

CONTAGEM PARALELA DE OBJETOS EM UMA MATRIZ BINÁRIA

Flood fill com Pthreads e consolidação por union-find

BETINA BORCHARDT KUSLER

SISTEMAS OPERACIONAIS - 2026/II - TURMA 330

PROF. FILIPO NOVO MÓR

O PROBLEMA

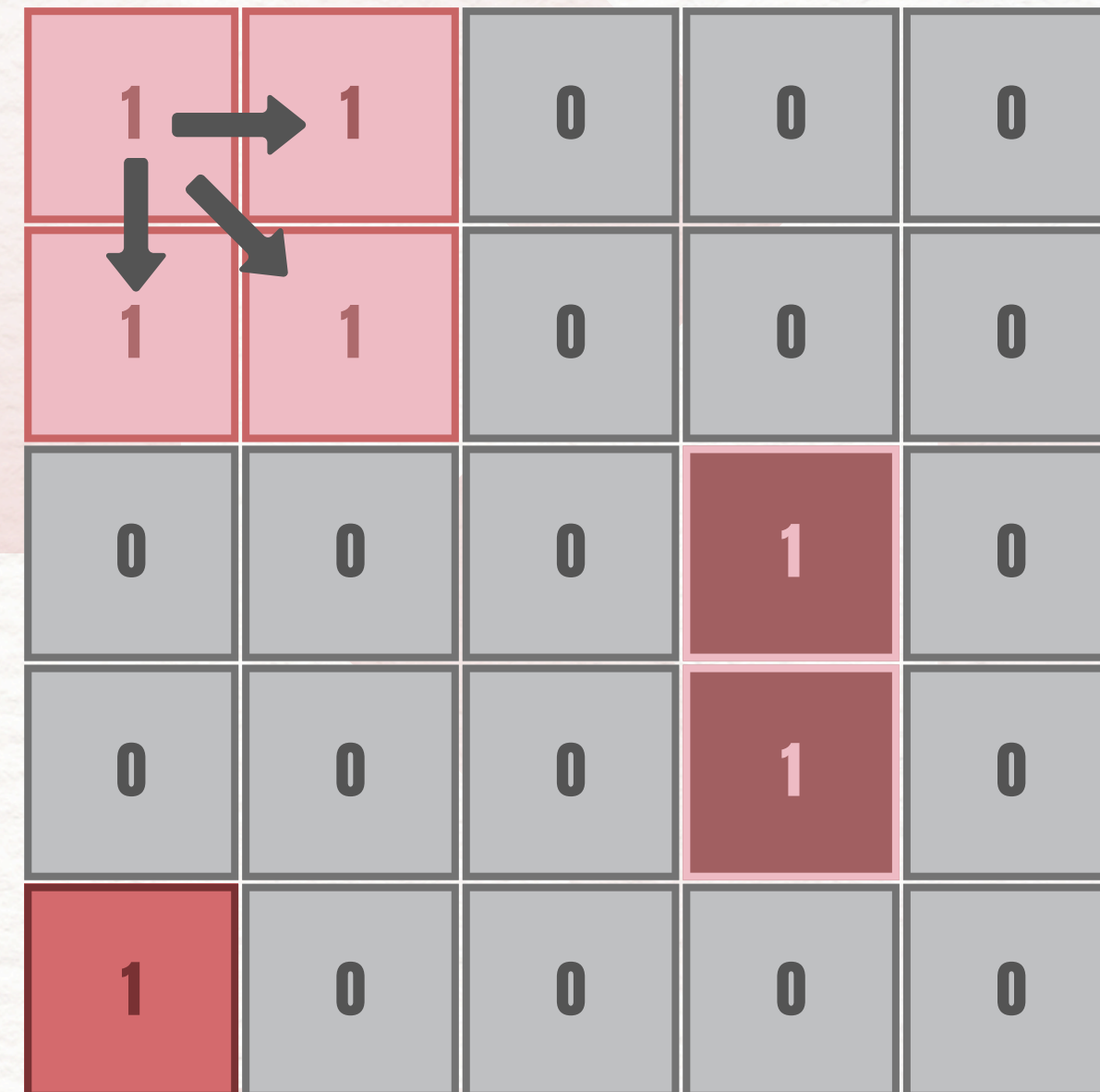
- Matriz binária: 0 = fundo, 1 = primeiro plano
- Objeto = grupo de células 1 conectadas
- Conectividade 8: horizontal, vertical e diagonal

NO	N	NE
0	CÉLULA	L
SO	S	SE

1	1	0	0	0
1	1	0	0	0
0	0	0	1	0
0	0	0	1	0
1	0	0	0	0

VERSÃO SEQUENCIAL: FLOOD FILL ITERATIVO

1. Percorre a matriz célula por célula
2. Achou um 1 não visitado: conta +1 objeto
3. Flood fill marca o objeto inteiro como visitado
 - Pilha explícita no lugar de recursão
 - Custo: $O(\text{linhas} \times \text{colunas})$



DIVIDIR E SOMAR NAO FUNCIONA

- Soma das contagens locais: $2 + 2 + 2 + 2 = 8$
- Resposta correta: 5
- O objeto central é contado 4 vezes

1	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	1	0
0	0	0	1	1	0	1	0
0	0	0	1	1	0	0	0
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	1
0	0	1	0	0	0	1	1

ESTRATÉGIA PARALELA: 3 FASES

Fase 1 (paralela) - Rotulação local: cada thread faz flood fill dentro do seu bloco

`pthread_join`

Fase 2 (paralela) - Fronteiras: procura objetos que cruzam blocos e os une

Fase 3 - Contagem: objetos = locais - uniões

Blocos distribuídos por fila dinâmica: 4 blocos por thread

SINCRONIZAÇÃO

DADO COMPARTILHADO	PROTEÇÃO
Fila de blocos (próxima tarefa)	mutex_fila
Union-find (vetor pai, contador de uniões)	mutex_uf
Vetor de rótulos	Nenhuma trava: cada thread só escreve no próprio bloco

Sem deadlock: nenhuma thread segura dois mutexes ao mesmo tempo

CONSOLIDAÇÃO DAS FRONTEIRAS

- Rótulo = Índice da célula semente + 1
(único, sem coordenação)
- Cada bloco verifica a fronteira inferior e a direita, com diagonais
- Rótulos vizinhos diferentes são unidos (union-find)
- 8 locais - 3 uniões = 5 objetos

1	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	23	0
0	0	0	28	29	0	23	0
0	0	0	36	37	0	0	0
0	0	0	0	0	0	0	0
0	0	51	0	0	0	0	56
0	0	51	0	0	0	56	56

CORREÇÃO

406



execuções, 0 falhas (14 matrizes * 8 configurações + repetições)

500



matrizes aleatórias iguais ao scipy, implementação independente

0



avisos do ThreadSanitizer

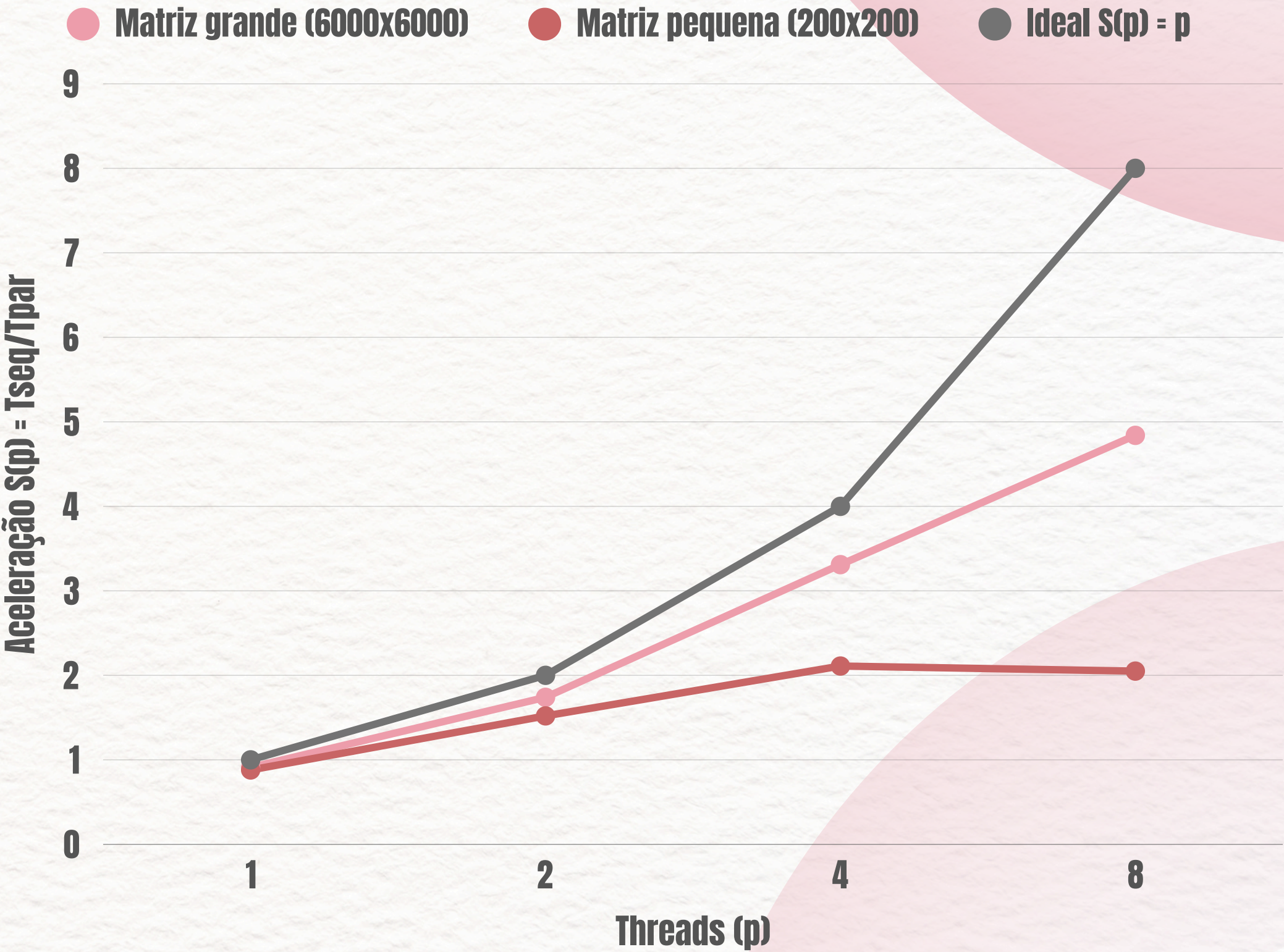
5 matrizes obrigatórias: 3, 4, 5, 6 e 7 objetos nas duas versões

DESEMPENHO

(matriz 6000 * 6000, Apple M2)

THREADS	TEMPO	ACELERAÇÃO
sequencial	728 ms	1,00
2	417 ms	1,74
4	220 ms	3,31
8	150 ms	4,84

ACELERAÇÃO EM RELAÇÃO À VERSÃO SEQUENCIAL



CONCLUSÕES

- Resultados idênticos nas duas versões, em todas as configurações
- Aceleração de 3,31 com 4 threads e 4,84 com 8
- Aprendizado: a sincronização se concentra numa etapa pequena, o cálculo pesado roda sem travas
- Melhoria futura: compactar rótulos para reduzir memória
- Ferramenta de IA usada na geração do código e da documentação, declarada no relatório
- github.com/borchardtkusler/SISOP-T1